

System Test Plan

Project: VeloPortal Bicycle Simulation and Rental Management Ecosystem

Candidate Name: Elvis Muthomi

Registration Number: 25/01220

Course: Diploma in Information Technology (DIT)

Supervisor: Madam Faith

Submission Date: June 2026

Contents

Chapter Seven: System Test Plan

7.1. Introduction

7.2. Testing Approach

7.3. Test Cases and Results

7.4. Bug Fixing and Debugging

7.5. User Feedback and Improvements

7.6. Appendices and References

Chapter Seven: System Test Plan

7.1. Introduction

7.1.1. Purpose: This section provides an overview of the VeloPortal system functionalities, ensuring they align with the requirements specified in the SRS and SDS documents [cite: 1, 3]. It serves as a blueprint for verifying the system's operational integrity, including the component compatibility simulator, rental scheduling engine, and administrative dashboard.

7.1.2. Testing Scope: The testing effort encompasses all functional modules, focusing on data processing logic, interface responsiveness, and security boundary enforcement. It covers unit-level logic checks, integration of sub-systems, and end-to-end user workflows.

7.2. Testing Approach

7.2.1. Testing Objectives: The primary goals are to identify defects early, ensure all functional requirements (REQ-1, REQ-2, REQ-3) perform as documented, and validate that the user experience is intuitive and stable across mobile and desktop viewports.

7.2.2. Testing Levels:

- **Strategy:** The project employs a hybrid approach. Automated testing is used for core logic (e.g., geometric fit calculations and booking scheduling), while manual testing is utilized for UI/UX evaluation and browser compatibility verification.
- **Types of Testing:**
 - **Unit Testing:** Testing individual functions, such as the geometry validation logic [cite: 3].
 - **Integration Testing:** Ensuring the front-end interacts correctly with the SQLite database and internal RESTful services.
 - **System Testing:** Validating the complete application workflow from user input to database record updates.
 - **User Acceptance Testing (UAT):** Verifying that the system meets the operational needs of storekeepers and student commuters.

7.3. Test Cases and Results

7.3.1. Test Case Details: Each test case is documented with a unique ID, description, preconditions, specific test steps, expected results, and recorded actual results to ensure repeatability and clarity.

7.3.2. Status and Notes: Every test is assigned a 'Pass' or 'Fail' status. Observations, including links to related defect reports in the tracking system, are documented to facilitate remediation.

7.4. Bug Fixing and Debugging

7.4.1. Bug Tracking: A standardized tool is utilized to manage defects, documenting a unique ID, bug description, severity level, priority, and clear reproduction steps.

7.4.2. Resolution Process: Team members are assigned to investigate and address identified bugs. The process involves logging resolution steps and updating the status once the fix has been verified through retesting.

7.4.3. Known Issues: A registry of unresolved bugs is maintained, detailing the issue, impact, and planned resolution schedule for future iterations.

7.5. User Feedback and Improvements

7.5.1. Feedback Collection: User input is gathered through structured surveys, direct interviews with shop attendants, and beta testing sessions with student commuters.

7.5.2. Feedback Summary: Feedback is categorized into three core domains: Usability (Ease of navigation), Functionality (Feature efficacy), and Performance (Load times and responsiveness).

7.5.3. Implemented Improvements: All modifications, patches, or UX adjustments made in response to user feedback are formally documented to maintain an audit trail of system evolution.

7.5.4. Future Recommendations: Suggestions for future project iterations—such as advanced analytics dashboards or automated inventory replenishment systems—are provided based on collected user insights.

7.6. Appendices and References

Testing Visuals: This section contains screenshots captured during testing, demonstrating successful UI interactions, error banner triggers, and system validation alerts.

References: The testing process is anchored by the System Requirements Specification (SRS), System Design Specification (SDS), and standard software engineering practices (Pressman, 2010; Kendall & Kendall, 2014) [cite: 1, 2, 3].